

EXAMEN PARCIAL

PROGRAMACIÓN ORIENTADA A OBJETOS II

Docente: Mg. Aldo H. Zanabria Gálvez

Duración: 90 minutos

Indicaciones: Responda con claridad. Justifique cuando se solicite. En las preguntas de programación, utilice código limpio, ordenado y correctamente estructurado.

SECCIÓN I: CONCEPTOS TEÓRICOS (10 puntos)

1. Defina el concepto de **polimorfismo** y mencione sus tipos.
2. Diferencie entre **herencia simple** y **herencia múltiple**.
3. ¿Qué es una **clase abstracta** y para qué se utiliza?
4. Explique el concepto de **encapsulamiento** con un ejemplo.
5. ¿Qué es el **enlace dinámico (late binding)**?
6. Diferencie entre **sobrecarga** y **sobrescritura** de métodos.
7. ¿Qué es una **interfaz** en términos conceptuales dentro de C++?
8. Explique el uso de la palabra clave **virtual**.
9. ¿Qué es el **principio de sustitución de Liskov**?
10. ¿Qué problema resuelve el uso de punteros a clases base?

SECCIÓN II: ANÁLISIS DE CÓDIGO (10 puntos)

11. Analice el siguiente código y explique la salida:

```
1      #include <iostream>
2      using namespace std;
3
4      class Base {
5      public:
6          virtual void mostrar() {
7              cout << "Base" << endl;
8          }
9      };
10
11     class Derivada : public Base {
```

```
12         public:
13         void mostrar() override {
14             cout << "Derivada" << endl;
15         }
16     };
17
18     int main() {
19         Base* obj = new Derivada();
20         obj->mostrar();
21         return 0;
22     }
23
```

12. ¿Qué sucede si se elimina la palabra clave `virtual` en el método `mostrar()`?

13. Identifique y explique el error del siguiente código:

```
1     class A {
2         public:
3         void metodo() = 0;
4     };
5
```

14. Analice el siguiente código y determine si existe polimorfismo. Justifique su respuesta.

```
1     #include <iostream>
2     using namespace std;
3
4     class Animal {
5     public:
6         void sonido() {
7             cout << "Animal hace sonido" << endl;
8         }
9     };
10
11     class Perro : public Animal {
12     public:
13         void sonido() {
14             cout << "Perro ladra" << endl;
15         }
16     };
17
```

15. ¿Cuál es la salida del siguiente programa?

```
1      #include <iostream>
2      using namespace std;
3
4      class A {
5      public:
6          A() { cout << "A"; }
7      };
8
9      class B : public A {
10     public:
11         B() { cout << "B"; }
12     };
13
14     int main() {
15         B obj;
16     }
17
```

16. Explique el posible problema de memoria en la siguiente instrucción:

```
1      Base* obj = new Derivada();
2
```

17. Analice si el siguiente código cumple con el principio de encapsulamiento:

```
1      class Cuenta {
2      public:
3          double saldo;
4      };
5
```

18. ¿Qué ocurre si un destructor no es virtual en un esquema de herencia?

19. ¿Qué función cumple la palabra clave `override` en C++?

20. Analice la siguiente clase y determine qué tipo de clase es:

```
1      class Figura {
2      public:
3          virtual double area() = 0;
4      };
5
```

SECCIÓN III: DESARROLLO DE CÓDIGO (10 puntos)

21. Cree una clase base `Vehiculo` con un método virtual `mover()`.

22. Cree dos clases derivadas: **Auto** y **Bicicleta**, que redefinan el método **mover()**.
23. Implemente un ejemplo de **polimorfismo usando punteros**.
24. Cree una clase abstracta **Figura** con el método virtual puro **area()**.
25. Implemente las clases **Rectangulo** y **Circulo** heredando de **Figura**.
26. Cree un ejemplo de **sobrecarga de funciones**.
27. Cree una clase con atributos privados y métodos públicos, aplicando correctamente el **encapsulamiento**.
28. Implemente un constructor y un destructor que muestren mensajes en pantalla.
29. Cree un ejemplo donde se utilice **late binding**.
30. Desarrolle un mini sistema con las siguientes características:
 - Clase base: **Empleado**
 - Clases derivadas: **Gerente**, **Obrero**
 - Método virtual: **calcularSalario()**

Criterios de evaluación:

- Claridad conceptual
- Correcta aplicación de los principios de POO
- Estructura lógica de las respuestas
- Sintaxis adecuada en los ejercicios de programación

Éxitos en el desarrollo de su evaluación